

App name: Study PDF Library

The problem

Most students have multiple PDFs for study materials that end up unorganized and scattered across different places, making it hard to keep track of which file is which and find what they need when they sit down to study.

Who is this for

Who: High school and college students juggling multiple subjects and courses, each with its own set of PDF readings, lecture slides, and scanned notes.

What they do instead: Rely on a general-purpose PDF reader, Google Drive folders, their phone's default file/downloads app, or physical printouts.

Core features (MVP)

#	Feature	Still in the MVP?	Flutter pieces it needs	Honest Estimate
1	Import PDF	Keep	FloatingActionButton, file picker, setState, document model, Drift insert	5 hours
2	View PDF	Keep	PDF viewer widget/package, AppBar, Scaffold, navigation, page controls	6 hours
3	Bookmark / progress tracking	Keep	PDF viewer, IconButton, setState, Drift queries/updates, page-change callback	5 hours
4	Organize into folders and sections	Keep	ListView.builder, Card, TextField, showDialog, Navigator.push, Drift queries	6 hours
5	Study Task Board	Keep	ListView.builder, Card, ExpansionTile (or ExpansionPanelList), PopupMenuButton (for status switching), showDialog, TextField, Drift updates	8 hours

Stretch goals

1. Color customization, including dark mode and other themes.
2. In-PDF text highlighting and other annotation features.
3. Advanced PDF organization or study features, if there is enough time after the MVP is complete.
4. One-tap AI PDF summarization using the Gemini API.

How my app saves data

If two different people install my app, should they see the same data?

No. Each user should have their own separate study materials, folders, bookmarks, reading progress, and study tasks. The app is designed as a personal study library, so there is no need for users to share or synchronize their data through an online server.

Roughly how many records does my app hold in a realistic week of use?

Around 30–100 records in a typical week of use. This is an estimate across documents, folders, bookmarks, reading progress, and study tasks. The actual number will depend on how many subjects and study materials the student uses.

My choice:

Drift (SQLite)

Why this one and not the others:

I chose Drift because my app needs to store several types of structured data, including documents, folders, bookmarks, reading progress, and study tasks. Unlike `shared_preferences`, which is better suited for simple settings and small pieces of data, Drift allows me to organize this information into related tables and query or update individual records. I accepted the additional setup and database code compared with simpler local storage options because it gives my app a more suitable structure as the number of saved records grows and supports the relationships and queries needed by my data.

What I save, concretely:

1. **Folder:** id, name
2. **Document:** id, fileName, filePath, folderId, dateAdded, lastPageRead
3. **Bookmark:** id, documentId, pageNumber, dateCreated
4. **Task:** id, title, status (To Study, In Progress, Revision, Completed)

Have I tried it yet?:

Yes. I completed a one-hour Drift spike using a minimal table, and it was code-generated and queried successfully in under a minute. The spike also showed that my Windows development machine does not currently have a working Android emulator, which is a setup task I need to resolve early regardless of the database choice.

One thing I want to add that the course did not teach

Feature: One-tap AI PDF summarization

Package/service: Gemini API, accessed through a Flutter-compatible API client.

Platform: The feature can be developed as part of the Flutter app, but it will require an internet access to the Gemini API.

Web fallback: If the AI feature cannot run in the browser, the PDF viewer will still work normally and the Summary button can display a sample/stand-in summary instead of making the API request. The real AI summarization can be demonstrated on a supported device or through a recording.

Core or stretch goal: Stretch goal. The app's main purpose is still organizing and reading study materials, so it remains fully usable without AI summarization. If the main MVP is completed early enough, I may implement this feature as an additional improvement.

How my project runs when someone else opens it

I am keeping the device_preview wrapper: Yes.

My app runs in a browser with flutter run -d web-server, start to finish, with every screen reachable: Not yet. The Study PDF Library has not been implemented yet, so browser testing will be done as development progresses.

Anything that needs real hardware degrades to sample data instead of crashing: Not applicable for the current MVP. The core features are intended to work without specialized hardware.

Public GitHub repository / secrets: The project will be kept in a public GitHub repository. The MVP doesn't require API keys or personal data. If the optional Gemini summarization feature is implemented, the API key will be kept server-side and outside the public repository.

Data the app remembers

Thing	Fields	Where it is saved
Document	id, fileName, filePath, folderId, dateAdded, lastPageRead	document table
Folder	id, name	folder table
Bookmark	id, documentId, pageNumber, dateCreated	bookmark table
Task	id, title, status	task table

Screens needed

1. Home screen / library
2. Courses / subject folders
3. PDF view
4. Kanban Board
5. Settings

Risks

Risk 1: PDF viewer and reading progress

The risk I named last time: Getting the PDF viewer to render pages correctly and perform smoothly, including page navigation, zoom, scrolling, and saving/restoring reading progress. This is still a significant risk. M4 and M5 gave me more experience with Flutter widgets and state management, but I have not yet implemented the PDF viewer, so I have not tested this risk directly.

First step: Test the chosen PDF viewer package with a few sample PDFs and implement basic page navigation and zoom before building the rest of the PDF-related features.

When: Week 1 of implementation.

Risk 2: Development environment / Drift

A new risk I did not see before: My Windows development machine currently does not have a working Android emulator. This could slow down testing of features that need to be checked on an Android device. Drift behaviours might break the app.

First step: Set up and test a working Android emulator, and research about drift and test how it behaves

When: Week 1-2 of implementation

.What changed and why

Section	Prelim said	Now says	Why it changed
Core features	The MVP had the main library, PDF viewing, organization, bookmarking/progress, and study task features.	The same five features remain in the MVP, but each now has specific Flutter pieces and an estimated implementation time, totaling 30 hours.	I kept the same scope after reviewing what I learned from M4/M5, AI suggestions, and research. Breaking each feature into its expected Flutter components made the workload easier to

			estimate and identify the Study Task Board as the highest-risk feature.
Data the app remembers	The data the app would remember was described more generally.	The revised proposal defines specific tables and fields for Documents, Folders, Bookmarks, and Tasks, using IDs to connect related records.	This makes it clearer how the app will save and retrieve related study information.
Stretch Goals	The prelim focused mainly on customization and PDF annotation ideas.	The revised proposal also includes one-tap AI PDF summarization using the Gemini API, along with advanced organization/study features.	I wanted to add a feature that could improve the app if the MVP is completed early. AI summarization is kept as a stretch goal so it does not put the core project at risk.
Risks	Getting the PDF viewer to work correctly and save reading progress.	The PDF viewer remains a major risk, and a lack of knowledge of Drift database and development-environment/database-testing risk was added because my Windows machine currently has no working Android emulator.	I added the information needed to make the project easier to reproduce and demonstrate.